

Simulate Before You Serve: Predicting a Decision-Model Cascade from Frozen Outputs, and Checking the Prediction on Real Hardware

Kostadis Roussos
Independent researcher
kostadis@gmail.com

October 2026

Abstract

A cascade answers with a cheap model and escalates uncertain items to an expensive one. Choosing the cheap model, the gate and the threshold usually means deploying and measuring each candidate. We show that when every candidate has already been run on a frozen evaluation set, the cascade can instead be *replayed* from the saved per-item answers, confidences and request times, and we check that replay against the real system. The setting is a working pipeline for tabletop role-playing campaign records. Five non-generative decision models are candidate first stages in front of a generative model (`qwen3.8-flash-next`) on four judgment tasks labelled by a game master, served on two NVIDIA DGX Spark (GB10) systems, with the goal of the generative model’s accuracy at lower latency. The replay evaluated the whole design space in 17.7 s of CPU time. It chose Decision-2.0-Nox-4B as the first stage for two tasks, Nox alone for a third, and no cascade for the fourth. We then ran that cascade live with thresholds frozen in advance. It held the generative model’s accuracy (the cascade-minus-qwen difference was predicted within 0.1 points on the three tasks with escalation) while answering $2.2\times$ and $1.8\times$ faster single-stream and with $2.4\times$ and $2.2\times$ the throughput under 8-way load. Escalation decisions were predicted exactly, because the decision model is bit-for-bit deterministic; the generative model at temperature 0 was not, changing 1–7% of its answers with load and serving configuration. Latency predictions were 3–4 $\times$ wrong when the replay mixed load regimes and within 5–9% when it did not. A two-station queueing model predicted throughput within 3% when one stage dominated and 14–25% high when the stages were balanced. A serving ablation found that, with speculative decoding off, prefix-cache hits on this model’s serving image were 38% slower and changed answers, a hidden variable for any replay.

1 Introduction

A practitioner who wants to put a cheap model in front of an expensive one faces a design space before any deployment question: which cheap model, which gate, which threshold, for which task. Each point in that space can be measured by deploying it, at the cost of GPU time, a serving change and a benchmark run per point. Many such points, however, are already determined by experiments that have been run. If every candidate model has been run on every item of a frozen evaluation set, and inference is deterministic, then the answer, the confidence and the request time of any cascade on any item are already recorded. The cascade can be *replayed* rather than run.

This paper tests that idea on a working pipeline. A companion study [1] ran six non-generative decision models and a generative model, `qwen3.8-flash-next`, on a frozen dataset of one game

master’s rulings over tabletop role-playing transcripts. Its practical verdict was that the generative model was the most accurate on most judgment steps and the decision models were an order of magnitude faster. vLLM Semantic Router’s *System One Auto* [3] suggests combining the two, by answering with a small decision model and escalating uncertain items. Our goal is narrow: *the generative model’s accuracy at lower latency*. We study it twice:

1. **Analytically**, by replaying the frozen outputs: five candidate first stages, four tasks, a held-out threshold that targets parity with the generative model, and latency and throughput composed from measured service times. The whole design space evaluates in seconds of CPU time.
2. **Empirically**, by running the cascade the replay recommends, Decision-2.0-Nox-4B in front of qwen, as a real two-box router on two NVIDIA DGX Spark (GB10) systems, with every threshold frozen before the first live request, single-stream and under 8-way load. We then repeat the qwen arms under three serving configurations, to find out what breaks the replay’s central assumption.

Findings. (i) The replay picked the right design. Nox-4B $\rightarrow$ qwen held qwen’s accuracy on triage and fact typing while answering $1.8\text{--}2.2\times$ faster single-stream and $2.2\text{--}2.4\times$ faster under load; Nox alone sufficed for entity typing; spelling should not be cascaded. The other four first stages were each too weak or too slow. (ii) Escalation decisions were predicted *exactly*, because the decision model is bit-for-bit deterministic. (iii) The cascade-minus-qwen accuracy difference was predicted within 0.1 points, although the generative model at temperature 0 did not reproduce 1–7% of its own answers. (iv) Latency predictions were $3\text{--}4\times$ wrong when the replay mixed load regimes and within 5–9% when it did not. A two-station queueing model predicted 8-client throughput within 3% when one stage dominates and overpredicted it by 14–25% when the two stages are balanced. (v) Turning speculative decoding off did not make the generative model reproducible; its answers depend on load and serving configuration together. With speculative decoding off and prefix caching on, cache hits on this model’s serving image became 38% slower and changed answers, which made the replay underpredict cascade latency by a quarter.

Why it matters. The replay turned the design question into a table computed in under a minute; the live check took several GPU-hours across two machines and a serving restart for each configuration tested. Where a replay is trustworthy, and where it quietly is not, is the practical content of this paper.

2 Background

2.1 The companion study and its frozen dataset

A companion paper [1] evaluated six non-generative *decision models* against a generative model on judgment steps of a pipeline that turns recorded tabletop role-playing sessions into campaign records. A decision model reads a **state** once and a small head returns a probability for every option of every typed question (**choice**, **noul**, **score**) through the SystemOne interface [2]; no text is generated. Labels are one game master’s (GM’s) own past rulings, rendered once into a read-only, versioned dataset with every model-facing **state** already built, so every model was run against the same bytes. That study treated the models as alternatives. Its conclusion for the choice tasks was mixed: the generative model, **qwen3.8-flash-next**, was usually the most accurate, while the

decision models were 10–25 $\times$ faster and sometimes as accurate. This paper asks whether the two can be *combined*.

We reuse four of its tasks, unchanged:

E2 spelling (497 items) choose the canonical spelling of a speech-recognition-mangled name from a candidate list, or **LEAVE/OTHER**.

E3 triage (864 tokens) is an unknown capitalised token a campaign name (forward it to the spelling step) or not (filler, rules term, real-world chatter)? 84 are real names. Here we score it as binary accuracy and separately count *missed names*, the error that matters: a dropped name is a silent deletion nothing downstream can recover.

E4 fact typing (1,691 facts) file an extracted fact under one of eight types.

E4 entity typing (92 GM merge rulings) the entity type of a dossier subject, five types.

2.2 Cascades and System One Auto

A cascade answers with a cheap model and re-asks an expensive one only when the cheap model is unsure [4, 5]. vLLM Semantic Router’s *System One Auto* [3] applies the idea to decision models: Decision-2.0-Kai-0.6B answers first, and a gate on its top probability ($\max(p, 1 - p)$ for yes/no questions) escalates the request to Decision-2.0-Vega-27B. On 231 public benchmark items, with a threshold frozen on a separate calibration set, the cascade scored 82.3% between Kai (65.8%) and Vega (87.9%) while escalating 46% of requests. Two differences matter for us. Our large model is *generative* and reached through a chat endpoint, so the two stages do not share an interface or a runtime; and our goal is narrower: *the large model’s accuracy at lower latency*, not a point between the two.

3 Method

3.1 The cascade

For each item the small model S (a decision model) returns an answer s and a gate confidence c : the top option probability for **choice** questions, and $\max(p, 1 - p)$ for the binary E3 reading, where $p = P(\text{campaign})$. If $c < t$ the same question goes to the large model B , prompted exactly as in the companion study (the option list rendered into a chat prompt, JSON answer parsed), and the cascade answers with B ’s answer; otherwise it answers s . B is **qwen3.8-flash-next** with reasoning disabled, on one DGX Spark (spark1); every S runs on the second (spark2). The two boxes are on the same LAN as the client.

3.2 The replay model

The companion study already ran every model on every item, at temperature 0 (generative) or with a deterministic head (decision models), and kept the per-item answers, confidences and request times. If inference is deterministic, the cascade’s answer on item i at threshold t is a pure function of those saved records:

$$\hat{a}_i(t) = \begin{cases} b_i & c_i < t \\ s_i & \text{otherwise} \end{cases}, \quad \hat{T}_i(t) = T_i^S + [c_i < t] T_i^B,$$

so accuracy, escalation rate and the full latency distribution of any (small model, threshold) pair can be computed without running a model. We call this the *replay model*. It makes two assumptions that the live experiment tests: **(A1) determinism**, that a model asked again returns the saved answer (and, for S , the saved confidence, since the gate depends on it); and **(A2) composable service times**, that a cascade request costs one S request plus, if escalated, one B request, each as long as when measured alone in the same load regime. A2 is a statement about load: the saved T^B came from an 8-way concurrent run and T^S from single-stream runs, so a naive replay mixes regimes. We therefore report latency from the replay with T^B re-measured single-stream (Section 3.4), and keep the naive figure only to show the size of that error.

For concurrent load the composition is no longer additive. We model the c -client closed loop as two stations visited with probabilities 1 (S) and p (B , the escalation rate), each with a response time $R(n)$ that depends on the number n of requests in it, linearly interpolated between its measured single-stream and 8-way means. Little’s law [6] at each station, $n_S = XR_S(n_S)$ and $n_B = XpR_B(n_B)$ with $n_S + n_B = c$, is solved for the throughput X , in the spirit of mean-value analysis with load-dependent stations [7]. We compare it with two simpler predictions: no contention, $X = c/\mathbb{E}[\hat{T}]$, and the bottleneck bound $X \leq \min(X_S^{\max}, X_B^{\max}/p)$.

3.3 Choosing the threshold

Our objective is parity with B at the lowest cost. On a training set the *parity gate* is the lowest threshold whose cascade accuracy is at least B ’s accuracy alone (and, on E3, that misses no more names than B); because escalating everything reproduces B exactly, such a threshold always exists. Thresholds are fitted 2-fold by group: sessions (E2, E3), chapters (E4 facts) or subjects (E4 entities) are sorted and split alternately, the gate is fitted on one half and applied to the other, so no item’s own label influences its threshold. Allowing a tolerance $\delta > 0$ below parity did not generalise in a pilot: a $\delta = 0.5$ point budget on the training half became a 1.3–1.6 point loss on the test half for E4 facts, so we use $\delta = 0$ throughout.

3.4 The live experiment

For the pair Nox-4B $\rightarrow$ qwen we froze the held-out parity gates for every item before any live run (file SHA-256 aa21ed3f...), together with the replay’s predicted escalation and answer per item. A small router (`cascade_live.py`) then ran, against the real endpoints and using the same question definitions, prompts and parsers as the companion study: S alone, B alone and the cascade, each single-stream ($c = 1$) and with 8 concurrent clients ($c = 8$), on all four tasks. Every request’s wall time is measured at the client, so cascade latency includes the router and both network hops. Live and replay results are compared item by item.

4 Results from the replay model

Table 1 is the replay model’s answer to the design question: for each of five decision models in front of qwen, and each task, the held-out parity gate’s accuracy, escalation rate and mean latency. Accuracy comes from the frozen per-item answers; latency composes each decision model’s single-stream request times from the companion study with qwen’s request times re-measured single-stream on the same items (Section 3.4). Producing the whole table, plus the threshold sweeps behind Figure 1, took 17.7 s of CPU time and 628,880 cascade-item evaluations, and no GPU time at all.

Task	Small model	Accuracy (%)			Δ vs qwen (pt) [95% CI]	Esc. (%)	Mean (ms)	Speed-up vs qwen
		small	qwen	cascade				
E3 triage	nox	79.2	84.7	84.7 (6/6)	+0.0 [-0.3, +0.3]	33	255	2.1×
	kai	81.9	84.7	85.2 (7/6)	+0.5 [+0.0, +1.0]	56	309	1.7×
	clef-flash	79.9	84.7	84.4 (6/6)	-0.3 [-0.9, +0.1]	50	398	1.3×
	clef	83.6	84.7	84.5 (6/6)	-0.2 [-0.9, +0.5]	11	491	1.1×
	lux	78.5	84.7	84.3 (6/6)	-0.5 [-0.9, +0.0]	60	457	1.2×
E4 fact typing	nox	83.6	88.3	88.2	-0.1 [-0.7, +0.5]	40	295	1.7×
	kai	63.6	88.3	87.8	-0.5 [-1.0, -0.1]	91	478	1.0×
	clef-flash	81.6	88.3	87.4	-0.9 [-1.4, -0.4]	66	474	1.1×
	clef	76.8	88.3	87.7	-0.6 [-1.1, -0.1]	52	741	0.7×
	lux	86.3	88.3	88.1	-0.2 [-0.7, +0.2]	49	387	1.3×
E4 entity typing	nox	89.1	87.0	89.1	+2.2 [-2.2, +6.5]	0	145	4.2×
	kai	85.9	87.0	84.8	-2.2 [-8.7, +4.3]	7	75	8.2×
	clef-flash	87.0	87.0	87.0	+0.0 [-5.4, +5.4]	0	193	3.2×
	clef	89.1	87.0	89.1	+2.2 [+0.0, +5.4]	0	651	0.9×
	lux	88.0	87.0	88.0	+1.1 [-2.2, +4.3]	0	213	2.9×
E2 spelling	nox	19.7	76.1	74.8	-1.2 [-2.4, -0.4]	98	1304	0.8×
	kai	15.3	76.1	75.5	-0.6 [-1.6, +0.2]	97	1111	1.0×
	clef-flash	30.0	76.1	75.1	-1.0 [-2.0, -0.2]	96	1309	0.8×
	clef	66.4	76.1	75.1	-1.0 [-2.8, +1.0]	58	1545	0.7×
	lux	31.4	76.1	75.9	-0.2 [-0.6, +0.0]	99	1424	0.8×

Table 1: Replay-model predictions for every decision model in front of qwen (reasoning off), with the held-out parity gate. “Esc.” is the share of items sent to qwen. Mean latency is per item, single-stream. For E3 the cascade column also gives missed real names (cascade/qwen) out of 84. CIs are paired bootstrap over items. The speed-up is qwen’s mean single-stream latency on the same items divided by the cascade’s.

Nox-4B is the only small model that pays off broadly. It holds parity on E3 (84.7% against qwen’s 84.7%, the same 6 missed names) with 33% escalated, a predicted 2.1× speed-up; and on E4 fact typing (88.2% against 88.3%, CI [-0.7, +0.5]) with 40% escalated, 1.7×. On E4 entity typing it needs no escalation at all: alone it is at least as accurate as qwen (89.1% vs 87.0%), so the “cascade” is Nox alone at 4.2×.

The other small models fail for two different reasons. Kai-0.6B is the fastest model (21 ms) but too weak on fact typing (63.6%): the gate must send 91% of items to qwen, and the cascade is no faster than qwen. Clef-27B is the opposite: accurate enough to escalate rarely on E3 (11%), but its own request (439 ms) costs most of a qwen call (528 ms), so it saves almost nothing; on fact typing the Clef cascade is *slower* than qwen alone. A first stage only helps if it is both much cheaper than the second and confident on a large share of items; the replay makes that trade-off visible per task, before anything is deployed.

Spelling (E2) should not be cascaded. Every decision model is far below qwen on choosing a canonical spelling (15–66% against 76.1%), so the parity gate escalates 58–99% of items and every cascade is slower than qwen alone. This reproduces the companion study’s finding that string-level judgments are where decision models fail.

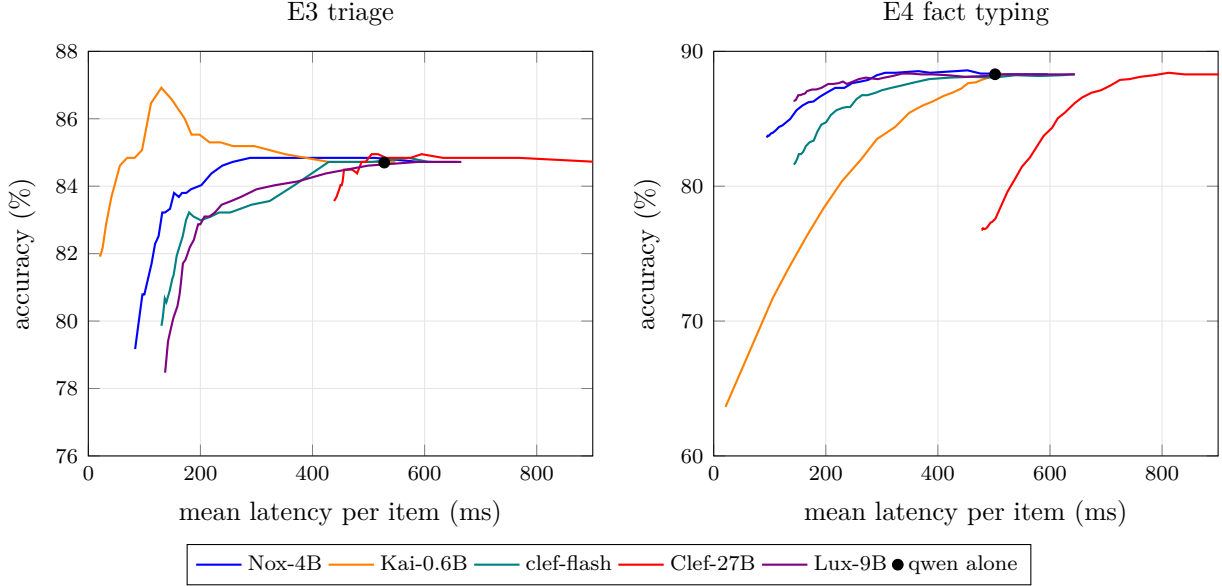


Figure 1: Replay-model accuracy–latency curves, sweeping the gate threshold from 0 (small model alone, left end of each curve) to 1 (everything escalated, converging on the qwen point). All items, not held out; single-stream latency. A useful cascade is a curve that reaches qwen’s accuracy well to the left of the qwen point. Kai’s E3 peak above qwen is the plain-accuracy artifact discussed in the text: it keeps confident not-a-name answers and misses real names.

No cascade beats qwen. Picking the threshold on the test items themselves (an upper bound) raises cascade accuracy by at most a few tenths of a point over qwen on E3 and E4 facts. The ceiling is set by the items both models get wrong: 124 of 864 on E3 and 140 of 1,691 on E4 facts. A cascade picks between two answers; it cannot produce a right answer neither model has. The binary E3 reading has one trap, noted here because the replay exposed it: a gate fitted for plain accuracy (rather than parity) finds a Kai threshold that “beats” qwen by keeping confident not-a-name answers while missing about half the real names. On this task that is the expensive error, which is why the parity gate also constrains missed names.

5 Results from the live cascade

Table 2 reports the Nox-4B $\rightarrow$ qwen cascade run for real with the pre-registered gates, against qwen alone run live on the same items in the same session, with qwen in its deployed configuration (MTP-2 speculative decoding, prefix caching on).

Accuracy parity held. On E3 the cascade matched qwen (84.5% vs 84.4%) and missed the same number of real names (5 vs 5). On E4 fact typing it was exactly level (87.8% vs 87.8%, CI [-0.6, +0.6]). On E4 entity typing Nox alone answered everything and was at least as accurate as qwen. On E2 the cascade trailed qwen ($\Delta = -1.2$ points), a loss the replay had predicted to the tenth of a point (next section).

Latency and throughput improved where the replay said they would. Single-stream, the cascade answered E3 $2.2\times$ and E4 facts $1.8\times$ faster than qwen on average (241 vs 528 ms and 278

Task	Accuracy (%), $c = 1$				Esc. (%)	Mean latency, $c = 1$			Throughput, $c = 8$ (items/s)		
	Nox	qwen	cascade	Δ [95% CI]		qwen	cascade	speed-up	qwen	cascade	gain
E3 triage	79.2	84.4	84.5	+0.1 [+0.0, +0.3]	33	528	241	2.19×	4.08	9.84	2.41×
E4 fact typing	83.6	87.8	87.8	+0.0 [-0.6, +0.6]	40	502	278	1.80×	4.07	8.79	2.16×
E4 entity typing	89.1	88.0	89.1	+1.1 [-2.2, +5.4]	0	616	139	4.43×	2.65	7.97	3.00×
E2 spelling	19.7	75.9	74.6	-1.2 [-2.4, -0.4]	98	1077	1227	0.88×	1.74	1.72	0.99×

Table 2: The live cascade (Nox-4B on spark2 $\rightarrow$ qwen on spark1) against qwen alone, measured in the same session. Accuracy and mean latency per item are single-stream ($c = 1$). Δ is cascade minus qwen with a paired bootstrap CI. Throughput uses 8 concurrent clients over the whole task. Latency is wall time at the client, including both network hops.

Task	Escalation		Answers		Δ acc. vs qwen (pt)		Mean latency, $c = 1$ (ms)			
	pred.	agree	agree	qwen repro	pred.	meas.	meas.	naive	replay	error
E3 triage	33.1%	100%	99.3%	99.0%	+0.0	+0.1	241	746	263	+9%
E4 fact typing	39.9%	100%	97.6%	96.9%	-0.1	+0.0	278	902	293	+5%
E4 entity typing	0.0%	100%	100.0%	98.9%	+2.2	+1.1	139	145	198	+42%
E2 spelling	98.0%	100%	93.2%	93.2%	-1.2	-1.2	1227	5080	1336	+9%

Table 3: Replay prediction against the live cascade ($c = 1$, qwen deployed configuration). *Escalation*: predicted share escalated, and the share of items where the live router made the predicted escalation decision. *Answers*: share of items where the live cascade’s answer equals the predicted one, and share where live qwen alone reproduces its own frozen answer. Δ *acc.*: cascade minus qwen, predicted (frozen answers) and measured (live run, same session). *Latency*: measured mean; the naive replay using the frozen request times (qwen’s were 8-way concurrent); the replay with single-stream service times measured live; and its relative error.

vs 502 ms), and E4 entities 4.4×. The distribution is bimodal: the median request is a Nox-only answer (82 ms on E3), while the 95th percentile is an escalated one (620 ms, against 618 ms for qwen alone). An escalated request costs both stages, so the cascade’s tail is slightly *worse* than qwen’s, as System One Auto also reported [3]. Under 8-way load the gain is larger: 2.4× (E3), 2.2× (E4 facts) and 3.0× (E4 entities) the throughput of qwen alone, because escalating 33–40% of items removes most of qwen’s queue. On E2, where 98% of items escalate, the cascade was slower than qwen alone single-stream (0.9×) and no faster under load (1.0×), as the replay predicted.

6 Prediction versus measurement

6.1 Accuracy: exact where the models are deterministic

Table 3 compares the replay’s item-level predictions with the live run.

Assumption A1 holds for the decision model, not for the generative one. Nox reproduced every frozen answer and every frozen confidence bit for bit (maximum absolute difference exactly zero across all 1691+864+92+497 items), so the live router escalated *exactly* the predicted items on every task. qwen, at temperature 0 in its deployed configuration, did not: it reproduced its own frozen answer on 99.0% of E3 items, 96.9% of E4 facts and only 93.2% of E2, where answers are longer. Every live-versus-predicted answer mismatch in the cascade, in every serving configuration we ran, is an escalated item on which qwen changed its answer; none comes from the gate. On E4

entity typing, where nothing escalates, the predicted and measured differences from qwen (+2.2 and +1.1 points) differ only because qwen alone changed one of its 92 answers. qwen’s single-stream and 8-way answers from the same session also agree on only 97.0% of E4 facts. Section 6.4 examines where it comes from.

The replay predicts the cascade’s *effect* better than its level. qwen’s drift moves the absolute accuracy of both qwen and the cascade together, so the replay’s absolute figure for E4 facts (88.2%) is half a point above the live one (87.8%). But the quantity the design decision rests on, the difference from qwen alone, was predicted within 0.1 points on E3, E4 facts and E2, including E2’s predicted *loss*. Both arms inherit the same qwen, so its noise largely cancels in the comparison.

6.2 Latency: right only in the right load regime

The naive replay, which uses the request times recorded in the companion study, predicted mean cascade latencies 3–4 $\times$ too high (746 against 241 ms on E3), because qwen’s recorded times came from an 8-way concurrent run while the cascade was measured single-stream. With service times measured in the target regime the same replay is within 5–9% on the three tasks with escalation, and predicts the speed-ups (2.0 $\times$ and 1.7 $\times$ predicted, 2.2 $\times$ and 1.8 $\times$ measured). The residual error is not router overhead, which measured 0.03 ms per request; it is drift in the models’ own service times between runs. qwen answered the escalated E4-facts items in 467 ms inside the cascade against 503 ms for the same items in the qwen-alone run an hour earlier; Section 6.4 tests whether prefix caching explains it. The largest error, +42% on E4 entities, came from the session’s very first run: Nox’s standalone E4-entity pass averaged 198 ms with a 95th percentile of 538 ms, against 139 ms inside the later cascade run. A replay is only as good as its service-time sample.

6.3 Throughput: a two-station model is close, and the naive one is not

Table 4 compares the measured cascade throughput at $c = 8$ with the three predictions of Section 3.2. Ignoring contention overpredicts throughput 3–5 $\times$. Both the bottleneck bound and the load-dependent two-station model are within 3% when one station dominates (E4 entities, where no item escalates, and E2, where nearly all do), and overpredict by 14–25% when the two are comparably loaded (E3, E4 facts). That is where such models are known to be loosest. The decision-model runtime is nearly serial (Nox alone gains only 8–43% throughput from 8 clients, against about 2 $\times$ for qwen), so at 33–40% escalation Nox and qwen saturate at similar rates. Clients then wait alternately on each stage, which a model built from per-station averages misses. The bottleneck bound still identifies the right design lever: on these tasks a faster or replicated first stage, not a faster qwen, is what would raise cascade throughput further.

6.4 Serving configuration and reproducibility

The deployed qwen uses two features that plausibly break assumptions A1 and A2: MTP speculative decoding, which verifies several drafted tokens in one forward pass and so changes the numerical path of a greedy decode; and automatic prefix caching (APC), which on this hybrid attention/linear-attention model restores recurrent state from cached blocks instead of recomputing it. We restarted qwen twice, with MTP off and then with MTP and APC both off, and repeated the qwen arm and the live cascade at both concurrency levels. The decision model and the gates were unchanged. Each configuration also got a second single-stream qwen pass over E3, identical to the first, except that with APC on every prompt is now a cache hit. Table 5 summarises.

Task	Esc.	Cascade throughput, $c = 8$ (items/s)				Prediction error		
		no contention	bound	model	measured	no cont.	bound	model
E3 triage	33%	30.4	12.31	11.86	9.84	+210%	+25%	+21%
E4 fact typing	40%	27.3	10.22	10.02	8.79	+210%	+16%	+14%
E4 entity typing	0%	40.5	7.92	8.10	7.97	+408%	-1%	+2%
E2 spelling	98%	6.0	1.77	1.75	1.72	+247%	+3%	+1%

Table 4: Cascade throughput at 8 concurrent clients: three analytical predictions against the measurement. “No contention” is $c/\mathbb{E}[\hat{T}]$ with single-stream service times; “bound” is $\min(X_S^{\max}, X_B^{\max}/p)$ from the standalone 8-client runs; “model” is the load-dependent two-station closed network of Section 3.2. Errors are relative to the measurement.

qwen config	Task	qwen alone				Δ vs qwen (pt)		Speed-up $c = 1$		Gain $c = 8$
		ms	repro	$c1=c8$	$c1=c1$	pred.	meas.	pred.	meas.	
MTP-2, prefix cache on (deployed)	E3	528	99.0	99.1	100.0	+0.0	+0.1	2.01	2.19	2.41
	E4 facts	502	96.9	97.0	—	-0.1	+0.0	1.71	1.80	2.16
	E2	1077	93.2	93.4	—	-1.2	-1.2	0.81	0.88	0.99
MTP off, prefix cache on	E3	605	99.0	99.5	99.8	+0.0	+0.0	2.09	1.58	2.71
	E4 facts	570	96.6	97.5	—	-0.1	+0.0	1.78	1.60	2.48
	E2	1736	94.4	94.4	—	-1.2	-2.0	0.87	0.93	1.01
MTP off, prefix cache off	E3	611	98.8	99.0	100.0	+0.0	+0.1	2.09	2.26	2.50
	E4 facts	574	97.6	97.0	—	-0.1	-0.1	1.78	1.86	2.06
	E2	1392	93.4	94.6	—	-1.2	-1.2	0.85	0.92	1.02

Table 5: The live check under three qwen serving configurations. *qwen alone*: mean single-stream latency (ms); share of items on which it reproduces its frozen answer (“repro”); agreement of its own single-stream and 8-client answers in the same session ($c1=c8$), and of two identical single-stream passes ($c1=c1$, E3 only). Δ : cascade minus qwen, predicted and measured. Speed-up: qwen-alone over cascade mean latency at $c = 1$, predicted by the replay (with that configuration’s service times) and measured. Gain: cascade over qwen-alone throughput at $c = 8$.

Answers: deterministic per configuration and load, not across them. In the deployed configuration two identical single-stream passes agreed on all 864 E3 items (100.0%), so qwen is repeatable when nothing changes. What changes its answers is the serving *context*. Single-stream and 8-client answers in the same session differ on 0.5–3% of E3 and E4-facts items and 5–7% of E2 items in every configuration, consistent with batch composition changing the reduction order of the kernels. The configuration itself also matters: single-stream answers with MTP on and off agree on 99.5% of E3 and 98.5% of E4 facts, and on only 95.0% of E2. Turning MTP off did not make qwen reproduce its frozen answers any better (E4 facts 96.6% vs 96.9%), nor did turning APC off as well (97.6%). The frozen answers were produced under one particular configuration and load (MTP-2, APC, 8-way), and no single-stream configuration recovers them. MTP is therefore not the cause of the drift; it is one of several things the answers depend on. In every configuration the replay’s predicted cascade-minus-qwen difference stayed within 0.1 points of the measurement on E3 and E4 facts.

Latency: prefix-cache hits are a hidden variable. With MTP off and APC on, the second, fully cached pass over E3 was *slower* than the first: 834 ms against 605 ms (+38%), and it changed

2 of 864 answers. In the deployed MTP-2 configuration the cached pass was instead slightly faster (519 vs 528 ms) with identical answers. The serving image carries a two-line fix for a prefix-cache bug on this model, in which a cache hit restored an all-zero recurrent state because the state slot was computed from the smallest KV-group block size (8 tokens at MTP-2) instead of the state block size. That fix was validated as bit-identical between cold and cached outputs *at MTP-2*. Our MTP-off result, where cache hits changed answers and ran 38% slower, is consistent with the fix not covering the block sizes that arise with MTP off. We did not investigate further. For a replay it means a serving flag nobody thinks of as affecting answers can change both answers and service times. It hit the cascade directly: every escalated prompt had just been sent by the qwen-alone run, so inside the MTP-off cascade qwen took 918 ms per escalated E3 item against 600 ms for the same items alone. The replay, built from cold-cache service times, underpredicted cascade latency by a quarter (290 against 383 ms) and overpredicted its speed-up ($2.1\times$ vs $1.6\times$). With APC off the effect disappears and the replay is again within 5–8% (292 vs 270 ms on E3).

The small residual in the other direction, qwen answering escalated items 3–7% faster inside the cascade than in a back-to-back qwen-alone run, persists with APC off (575 vs 608 ms), so it is not caching. It coincides with the cascade’s spacing of qwen requests (each is preceded by a Nox call and often by several non-escalated items). We did not isolate it further.

MTP is worth keeping for this workload. Even though every answer here is a short JSON object, MTP-2 cut qwen’s single-stream latency from 611 to 528 ms on E3 and from 1392 to 1077 ms on E2, whose answers are longer. The deployed configuration is the fastest and the best-behaved of the three for the cascade.

7 Discussion

What the replay is good for. It answered the design question correctly and cheaply. Of twenty (first stage, task) pairs it singled out one pair to deploy on two tasks, one task needing no cascade and one task to leave alone, and every one of those calls survived the live test. It also explained *why* the other first stages fail (too weak, so everything escalates; or too slow, so nothing is saved), which a single live run of one configuration would not have shown. Its accuracy-difference predictions were within 0.1 points. Exploring the space live would have meant running each candidate decision model on the GB10, re-serving it for each experiment, and measuring five cascades on four tasks under two load levels. In the replay that is a loop over saved records.

What it silently assumes. Three assumptions did the work, and each was violated somewhere:

1. *The gate’s inputs are reproducible.* They were, exactly, because a decision model’s head is a deterministic function of the input. This is the property that makes the cascade’s routing predictable, and it is a reason to put a decision model, rather than a sampled generative judge, at the gate.
2. *The expensive model’s answers are reproducible.* They were not, at the 1–7% level, even at temperature 0. The replay absorbed this because both arms of the comparison inherit the same model, but a replay that reports absolute accuracy should carry the run-to-run spread of the generative model as an error bar. The ablation shows this drift is not one feature’s fault: no single-stream configuration we tried, with or without speculative decoding and prefix caching, reproduced the frozen answers better than another. The answers are a function of model, serving configuration *and* load.

3. *Service times are measured in the regime being predicted.* Mixing an 8-way concurrent sample with single-stream ones produced a 3–4 $\times$ error that looked plausible: it was the latency we had first quoted when proposing the cascade. Throughput under load needs a queueing model, and the simplest one that respects per-station saturation was within 25%, while “no contention” was off by 3–5 $\times$.

Practical recipe. Freeze the evaluation inputs; run every candidate once on all of them, recording answer, confidence and request time, with each model’s service time measured in the load regime it will serve in; replay the cascade over the records with held-out thresholds targeting the objective you actually have (here parity, with the expensive error class constrained separately); then validate the chosen point live, with the gates frozen beforehand, and compare item by item. The item-level comparison is what separates “the prediction was right” from “the errors happened to cancel”.

Local hardware. Two observations are specific to running both stages on DGX Sparks. The decision-model runtime is nearly serial on the GB10, so under load the 4B first stage, not the generative model, sets the cascade’s throughput ceiling; replicating it is cheap in memory (~ 19 GB) and is the next lever. The cascade also turns two boxes into a pipeline: every item the decision model settles on spark2 in about 90 ms is a half-second generative request that spark1 never queues.

8 Limitations

- **One live pair.** Only Nox-4B $\rightarrow$ qwen was run live; the other four first stages are replay-only. Their accuracy predictions rest on the same determinism that held for Nox (all are deterministic-head decision models from the companion study), but their latency predictions use the companion study’s service times and were not re-measured.
- **Evaluated on the dataset the gates were fitted on.** Thresholds were fitted 2-fold by session, so no item’s label set its own threshold, but the live run used the same frozen items as the replay. That is deliberate (it isolates prediction error from sampling error), but it is not a test on new data. The two folds’ thresholds differ (0.70 vs 0.72 on E4 facts, 0.83 vs 0.88 on E3), and a deployment would need its own calibration set, as System One Auto notes [3].
- **One hardware and software profile.** Two GB10 systems on one LAN, one vLLM build, one decision-model runtime. The serial behaviour of that runtime under concurrency shapes the throughput results and may not hold for other runtimes.
- **Small label sets for some tasks.** E4 entity typing has 92 items; its CIs span ± 5 points.
- **The E3 metric is ours.** E3 is scored here as binary accuracy with a missed-name constraint, not with the companion study’s forwarded-share-at- k -misses metric.
- **Non-reasoning qwen only.** The thinking variant, which is the only candidate that could raise the accuracy ceiling, was not tested as a second stage.

9 Conclusion

Replaying frozen model outputs is a cheap and, within its assumptions, accurate way to design a cascade. Here it chose a two-box decision-model/generative-model cascade that kept the generative model’s accuracy at roughly twice its speed. It also predicted that accuracy difference within a

tenth of a point and routed every item exactly as the live system did. The assumptions are what a practitioner has to check: a deterministic gate (decision models provide one), a second stage whose answer drift is measured rather than assumed away, and service times sampled in the load regime and cache state being predicted. Get the last one wrong and a replay that looks authoritative is off by a factor of three. Measure them properly and a few seconds of arithmetic stands in for an afternoon of GPU time.

Data and code availability

The replay model (`clef/eval/cascade.py`), the live router (`cascade_live.py`), the script that generates every number, table and figure in this paper from the result files (`cascade_paper.py`) and the frozen dataset’s manifest are in the public repository [8]. The campaign data itself is not released because it quotes play sessions verbatim and names real people.

Use of AI assistance

The experiments, analysis and a first draft of this paper were carried out by an AI coding agent (Claude, Anthropic) working under the author’s direction, with the author reviewing results and making the decisions recorded in the paper. The author takes full responsibility for the content.

References

- [1] K. Roussos. Decision models at the game table: where non-generative classifiers help and fail in LLM tooling for tabletop role-playing campaigns. Manuscript, October 2026. In [8], `paper/main.tex`.
- [2] TypeSafe. Jev and the System One API. Documentation, <https://docs.typesafe.ai>, accessed October 2026.
- [3] vLLM Semantic Router Team. System One Auto. Blog post, <https://vllm-sr.ai/blog/system-one-auto/>, accessed October 2026.
- [4] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176, 2023.
- [5] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665, 2024.
- [6] J. D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.
- [7] M. Reiser and S. S. Lavenberg. Mean-value analysis of closed multichain queuing networks. *Journal of the ACM*, 27(2):313–322, 1980.
- [8] K. Roussos. dgx-fun: experiments with local LLM serving on NVIDIA DGX Spark. <https://github.com/kostadis/dgx-fun>, 2026.